545 Course Project Runbook

Table of contents

1. Extracting Data from Web
2. Configuration Setup
3. Daily Data Pull
4. Daily Data Load to PostgreSQL
5. Data from PostgreSQL to Python
6. Sales Dashboard Creation

1. Extracting Data from Web

Purpose: This script downloads sales data from an external source and updates a local CSV file. It ensures that duplicate records are removed and that the latest data is preserved.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
import requests
import pandas as pd
import os
```

- requests: To download the data from a remote URL.
- pandas: For data manipulation.
- os: To check if the file already exists.

Step 1: Download and Load the Data

The script fetches the dataset from an external URL and executes it to create a DataFrame named data.

```
url = 'http://ballings.co/data.py'
exec(requests.get(url).content)
```

Step 2: Define the File to Store Data

Specify the local CSV file path where the data will be saved:

```
file_path = '/Users/oliviahelms/ballings_group_777/sales_data2.csv'
```

Step 3: Check if the File Exists

- **If the file exists:**
    1. Load existing data.
    2. Combine existing and new data.
    3. Remove duplicates based on ['salesdate', 'productid', 'region']
    4. Save the updated data back to the CSV.
- **If the file does not exist:**
    1. Save the new data directly.

```
if os.path.exists(file_path):
    existing_data = pd.read_csv(file_path)
    combined = pd.concat([existing_data, data], ignore_index=True)
    combined.drop_duplicates(subset=['salesdate', 'productid', 'region'], keep='last', inplace=True)
    combined.to_csv(file_path, index=False)
else:
    data.to_csv(file_path, index=False)
print(f"Data saved to {file_path}")
```

2. Configuration Setup

Purpose: The purpose of this section is to define and document all environment variables, directory structures, configuration settings, and file-naming conventions required for the system to run. These settings ensure that:

- Sensitive credentials are loaded securely from the environment.
- All data acquisition tasks save files consistently and predictably.
- The PostgreSQL database connection is correctly configured.
- Downstream tasks (e.g., exporting DB snapshots) follow standardized directory and naming formats.

This configuration layer must be fully functional before any extraction, transformation, or loading processes can be executed.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
import os
from dotenv import load_dotenv
from datetime import datetime
```

- os: To check if the file already exists.
- dotenv: To load varibales.
- datetime: To work with dates.

Step 1: Load Environment Variables

```
load_dotenv()
```

Step 2: Get Data

This section sets up the daily data download. Data is retrieved from the DATA_URL endpoint and saved in the local DATA_DIR folder, which is created automatically if missing. Each file is named with a date-stamped template (raw_data_MM-DD-YYYY.pkl) to ensure unique, traceable daily downloads. These settings must be configured before running any data extraction.

```
DATA_URL = "https://ballings.co/data.py"

# Folder where the raw extracted pickle files will be saved locally
# Matches your friend's style → DATA_DIR
DATA_DIR = os.path.join("data", "raw_downloads")

# Ensure the directory exists
os.makedirs(DATA_DIR, exist_ok=True)

# File naming pattern with date (mm-dd-yyyy.pkl)
# Matches friend's use of "TEMPLATE" naming
DATA_FILENAME_TEMPLATE = f"raw_data_{datetime.now().strftime('%m-%d-%Y')}.pkl"
```

Step 3: Database Connection

This section defines the PostgreSQL database connection settings. The DB_CONFIG dictionary pulls host, port, database name, username, and password from environment variables, with sensible defaults if

any are missing. These credentials are required for connecting to the local or remote database and must be correctly set before running any database-related tasks.

```
DB_CONFIG = {
  "host": os.getenv("DB_HOST", "localhost"),
  "port": str(os.getenv("DB_PORT", 5432)),
  "dbname": os.getenv("DB_NAME", "postgres"),
  "user": os.getenv("DB_USER", "postgres"),
  "password": os.getenv("DB_PASSWORD")   # From .env file
}
```

Step 4: Export Configuration

This section sets up the local directory for database pickle exports (PICKLE_EXPORT_DIR), creating it automatically if it doesn't exist. Exported files use a timestamped naming template (PICKLE_EXPORT_TEMPLATE) in the format db_dump_YYYYMMDD_HHMMSS.pkl to ensure each dump is unique. The database password is loaded from environment variables for validation purposes.

```
PICKLE_EXPORT_DIR = os.path.join("data", "database_exports")

os.makedirs(PICKLE_EXPORT_DIR, exist_ok=True)

PICKLE_EXPORT_TEMPLATE = f"db_dump_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pkl"

password = os.getenv("DB_PASSWORD")

print(password)
```

3. Daily Data Pull

Purpose: Fetches the Ballings dataset from DATA_URL, validates it as a DataFrame, and saves it as a date-stamped pickle in DATA_DIR. Ensures the directory exists and reports success or errors.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
import requests
import pickle
import pandas as pd
import os
import sys
```

Step 1: Configuration Import

mports DATA_URL, DATA_DIR, and DATA_FILENAME_TEMPLATE from config.py for use in the data fetch workflow.

```
from config import DATA_URL, DATA_DIR, DATA_FILENAME_TEMPLATE
```

Step 2: Fetch and Save Daily Data

Retrieves the daily Ballings dataset from DATA_URL, executes it to produce a pandas DataFrame named data, validates the result, ensures the local DATA_DIR exists, and saves the DataFrame as a date-stamped pickle file using DATA_FILENAME_TEMPLATE. Reports success or errors and exits on failure.

```
def fetch_and_save_data():
    """
    Fetches the Ballings data (which creates a 'data' DataFrame),
    then saves it locally as a pickle file using the filename
    template defined in the config file.
    """

    print(f"Starting data retrieval from {DATA_URL}...")

    try:

        response = requests.get(DATA_URL)
        response.raise_for_status()

        local_vars = {}
        exec(response.content, local_vars)

        data_obj = local_vars.get("data")

        if data_obj is None or not isinstance(data_obj, pd.DataFrame):
            print("Error: The Ballings script did not produce a valid DataFrame named 'data'.")
            sys.exit(1)

        os.makedirs(DATA_DIR, exist_ok=True)

        save_path = os.path.join(DATA_DIR, DATA_FILENAME_TEMPLATE)

        with open(save_path, "wb") as f:
            pickle.dump(data_obj, f)

        print(f"Success: Saved {len(data_obj)} records to {save_path}")
```

```
    return save_path

except requests.exceptions.RequestException as e:
    print(f"Network error while retrieving data: {e}")
    sys.exit(1)
except Exception as e:
    print(f"Unexpected error: {e}")
    sys.exit(1)
```

Step 3: Script Execution Entry Point

Allows the script to be run directly from the command line, triggering the fetch_and_save_data() function to perform the full data retrieval and save workflow.

```
if __name__ == "__main__":
  fetch_and_save_data()
```

4. Daily Data Load to PostgreSQL

Purpose: Finds the most recent pickle file in DATA_DIR, loads it into a Pandas DataFrame, and inserts the records into the local PostgreSQL database table data_daily using the predefined DATA_SCHEMA. Validates file existence, DataFrame integrity, and reports success or errors during the process.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
import pandas as pd

import pickle

import os

import glob

import sys

from sqlalchemy import create_engine, types

from sqlalchemy.engine import URL
```

Step 1: Database and Data Directory Configuration

Imports database connection settings (DB_CONFIG) and the local data directory path (DATA_DIR) from config.py for use in data loading and insertion workflows.

```
from config import DB_CONFIG, DATA_DIR
```

Step 2: Daily Data Schema Definition

Defines the column names and data types for the daily Ballings dataset, mapping each field (date, product_id, quantity, price, customer_id) to the appropriate SQLAlchemy type for database insertion.

```
DATA_SCHEMA = {
  'date': types.DateTime(),
  'product_id': types.Integer(),
  'quantity': types.Integer(),
  'price': types.Float(),
  'customer_id': types.Integer(),
}
```

Step 3: Find Latest Pickle File

Searches the specified data_directory for all pickle files and returns the most recently modified one. Exits with an error if no pickle files are found, ensuring downstream processes use the latest available data.

```
def find_latest_data_file(data_directory):
  """
  Finds the most recently saved pickle file in the data directory.
  Matches friend's logic exactly: glob → check → max() by modification time.
  """
  search_path = os.path.join(data_directory, "*.pkl")
  files = glob.glob(search_path)

  if not files:
      print(f"Error: No pickle files found in {data_directory}")
      sys.exit(1)

  latest_file = max(files, key=os.path.getmtime)
  print(f"Found latest data file: {latest_file}")
  return latest_file
```

Step 4: Load Pickle File into DataFrame

Loads a pickle file from file_path into a Pandas DataFrame, validates that the object is a proper DataFrame, prints the total record count, and exits with an error if loading fails or the object is invalid.

```
def load_data(file_path):
  """
```

```
    Loads a pickle file containing a Pandas DataFrame.
    Matches friend's strict checking and prints record count.
    """
    try:
        with open(file_path, "rb") as f:
            df = pickle.load(f)

        if not isinstance(df, pd.DataFrame):
            raise TypeError("Loaded object is not a Pandas DataFrame.")

        print(f"Successfully loaded data. Total records: {len(df)}")
        return df

    except Exception as e:
        print(f"Error loading data from {file_path}: {e}")
        sys.exit(1)
```

Step 5: Insert DataFrame into PostgreSQL

Inserts a Pandas DataFrame into the specified PostgreSQL table using SQLAlchemy, applying the provided dtype_map for column types. Appends records to the table, prints the number of inserted rows, and exits with an error if the insertion fails.

```
def insert_records(df, table_name, dtype_map):
    """
    Inserts the DataFrame into a PostgreSQL database using SQLAlchemy.
    Follows the same style and function structure as your friend's code.
    """
    try:
        url = URL.create(
            drivername="postgresql",
            username=DB_CONFIG["user"],
            password=DB_CONFIG["password"],
            host=DB_CONFIG["host"],
            port=int(DB_CONFIG["port"]),
            database=DB_CONFIG["dbname"]
        )

        engine = create_engine(url)

        df.to_sql(
            table_name,
            engine,
```

```
        if_exists="append",
        index=False,
        dtype=dtype_map
    )

    print(f"Successfully inserted {len(df)} records into table '{table_name}'.")

except Exception as e:
    print(f"Database insertion failed: {e}")
    sys.exit(1)
```

Step 6: Main Execution Workflow

When the script is run directly, it finds the most recent pickle file in DATA_DIR, loads it into a Pandas DataFrame, and inserts the records into the PostgreSQL table data_daily using the defined DATA_SCHEMA. This orchestrates the full daily data load process from file to database.

```
if __name__ == "__main__":
  latest_file = find_latest_data_file(DATA_DIR)
  df = load_data(latest_file)
  TARGET_TABLE = "data_daily"
  insert_records(df, TARGET_TABLE, DATA_SCHEMA)
```

5. Extracting Data from Web

Purpose: A quick guide to connecting Python to PostgreSQL, running SQL queries, and loading results into pandas for fast data analysis.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
import psycopg2

import pandas as pd

from config import DB_CONFIG
```

Step 1: Build PostgreSQL Connection String

Constructs a secure connection string using values from DB_CONFIG, allowing Python to connect to the PostgreSQL database.

```
pgConnectString = (
  f"host={DB_CONFIG['host']} "
  f"port={DB_CONFIG['port']} "
  f"dbname={DB_CONFIG['dbname']} "
  f"user={DB_CONFIG['user']} "
  f"password={DB_CONFIG['password']}"
)
```

Step 2: Execute Query and Retrieve Data

Attempts to connect to PostgreSQL, run a SQL query, load the results into a pandas DataFrame, and safely handle errors while ensuring the database connection is closed.

```
try:
  pgConnection = psycopg2.connect(pgConnectString)
  query = "SELECT * FROM data_daily LIMIT 50;"
  df = pd.read_sql_query(query, pgConnection)
  print("\nSUCCESS! Retrieved rows:")
  print(df.head())
except Exception as e:
  print("❌ Error reading from PostgreSQL:", e)

finally:
  if 'pgConnection' in locals():
    pgConnection.close()
```

6. Sales Dashboard Creation

Purpose: This script loads sales data from a PostgreSQL database and builds an interactive Dash dashboard featuring KPIs, filters, and visualizations. It enables users to explore sales trends, regional performance, and product-level insights in real time.

Step 0: Import Required Libraries

The script imports necessary Python modules:

```
from dash import Dash, html, dcc, Input, Output

import plotly.express as px

import pandas as pd

import datetime as dt
```

```
import psycopg2

from config import DB_CONFIG
```

Step 1: Load Sales Data from PostgreSQL

Builds a database connection, reads the data_daily table into a pandas DataFrame, converts the sales date column, and returns the cleaned dataset.

```
def load_data():

  pg_string = (

      f"host={DB_CONFIG['host']} "

      f"port={DB_CONFIG['port']} "

      f"dbname={DB_CONFIG['dbname']} "

      f"user={DB_CONFIG['user']} "

      f"password={DB_CONFIG['password']}"

  )



  conn = psycopg2.connect(pg_string)

  query = "SELECT * FROM data_daily;"

  df = pd.read_sql_query(query, conn)

  conn.close()



  df["salesdate"] = pd.to_datetime(df["salesdate"], errors="coerce")

  return df
```

Step 2: Create KPI Card

Creates a styled dashboard card that displays a KPI title and value using Dash HTML components.

```
def kpi_card(title, value):

  return html.Div(

      style={

          "backgroundColor": "white",

          "padding": "20px",

          "borderRadius": "8px",

          "width": "22%",

          "textAlign": "center",

          "boxShadow": "0 2px 4px rgba(0,0,0,0.1)",

      },

      children=[html.H4(title), html.H2(value, style={"color": "#c21299"})],

  )
```

Step 3: Dash App Setup

Creates the Dash app instance that serves as the foundation for building and running the interactive dashboard.

```
app = Dash(name="Sales_Dashboard")
```

Step 4: Dash App Setup

Loads sales data, calculates KPIs, and constructs the full Dash interface—including header, status messages, filters, KPI cards, and placeholders for visualizations.

```
def serve_layout():

  df = load_data()

  total_items = df["itemssold"].sum()
  avg_discount = df["discount"].mean().round(2)
```

```
free_ship_rate = f"{df['freeship'].mean() * 100:.2f}%"

last_date = df["salesdate"].max()
days_old = (pd.Timestamp.today().date() - last_date.date()).days
pipeline_status = "Pipeline Fresh" if days_old <= 1 else "Pipeline Stale"

return html.Div(
    style={"padding": "20px", "font-family": "Arial, sans-serif"},
    children=[

        html.Div(
            style={
                "backgroundColor": "#c21299",
                "padding": "15px",
                "borderRadius": "6px",
                "color": "white",
                "marginBottom": "20px",
            },
            children=[html.H1("Sales Monitoring Dashboard", style={"textAlign": "center"})],
        ),

        html.Div(f"Data last loaded on: {last_date.date()}", style={"textAlign": "center"}),
        html.Div(f"Pipeline Status: {pipeline_status}", style={"textAlign": "center"}),
        html.Br(),

        html.Div(
            style={"display": "flex", "justifyContent": "space-around"},
            children=[
                kpi_card("Total Items Sold", f"{total_items:,}"),
                kpi_card("Average Discount", avg_discount),
                kpi_card("Free Shipping Rate", free_ship_rate),
            ],
        ),

        html.Br(),

        html.Div(
            style={
                "backgroundColor": "white",
                "padding": "15px",
                "borderRadius": "8px",
                "marginBottom": "20px",
```

```
            "boxShadow": "0 2px 4px rgba(0,0,0,0.1)",
            "width": "40%",
        },
        children=[
            html.H4("Filters"),
            html.Label("Region:"),
            dcc.Dropdown(
                id="region-filter",
                options=[{"label": r.upper(), "value": r} for r in sorted(df["region"].unique())],
                value=None,
                clearable=True,
            ),
            html.Br(),
            html.Label("Product ID:"),
            dcc.Dropdown(
                id="product-filter",
                options=[{"label": p, "value": p} for p in sorted(df["productid"].unique())],
                value=None,
                clearable=True,
            ),
        ],
    ),

    dcc.Graph(id="items-trend"),
    dcc.Graph(id="region-product-bar"),
    dcc.Graph(id="productid-items-bar"),
  ],
)

app.layout = serve_layout
```

Step 5: Generate and Update Dashboard Charts

Filters the dataset by selected region and product, then creates three interactive charts: a line chart for daily items sold, a bar chart for sales by region, and a bar chart for items sold per product ID, all styled and ready for display in the dashboard.

```
@app.callback(
  Output("items-trend", "figure"),
  Output("region-product-bar", "figure"),
  Output("productid-items-bar", "figure"),
```

```
    Input("region-filter", "value"),
    Input("product-filter", "value"),
)
def update_graphs(region, product):

    df = load_data()

    if region:
        df = df[df["region"] == region]

    if product:
        df = df[df["productid"] == product]

    trend = df.groupby("salesdate")["itemssold"].sum().reset_index()

    fig_trend = px.line(
        trend,
        x="salesdate",
        y="itemssold",
        title="Daily Items Sold Over Time",
        markers=True,
    )
    fig_trend.update_traces(line=dict(color="#ffc6e5"), marker=dict(color="#ffc6e5", size=8))
    fig_trend.update_layout(
        xaxis_title="Sales Date",
        yaxis_title="Items Sold",
        plot_bgcolor="white"
    )

    region_sales = df.groupby("region")["itemssold"].sum().reset_index()

    fig_bar = px.bar(
        region_sales,
        x="region",
        y="itemssold",
        title="Sales by Region",
        text_auto=True
    )
    fig_bar.update_traces(marker_color="#ff4fa3")
    fig_bar.update_layout(
        xaxis_title="Region",
        yaxis_title="Items Sold",
```

```
    plot_bgcolor="white"
)

prod_sales = df.groupby("productid")["itemssold"].sum().reset_index()

fig_bar2 = px.bar(
    prod_sales,
    x="productid",
    y="itemssold",
    title="Product ID vs Items Sold",
    text_auto=True
)
fig_bar2.update_traces(marker_color="#c21299")
fig_bar2.update_layout(
    xaxis_title="Product ID",
    yaxis_title="Items Sold",
    plot_bgcolor="white"
)

return fig_trend, fig_bar, fig_bar2
```

Step 6: Run Server

Starts the Dash server in debug mode, allowing the interactive dashboard to run locally and automatically reload on code changes.

```
if __name__ == "__main__":
  app.run(debug=True)
```